

Tablero Inteligente OSC NL

Implementación de la propuesta de transformación digital

Registro Estatal de Organizaciones de la Sociedad Civil
Secretaría de Igualdad e Inclusión · Gobierno de Nuevo León

Septiembre de 2026

1. La propuesta implementada

El Registro Estatal de Organizaciones de la Sociedad Civil concentra el padrón de las OSC de Nuevo León. Antes de este proyecto, ese padrón vivía en hojas de cálculo: cada consulta —cuántas organizaciones operan en un municipio, a cuáles les falta documentación, quién revisó qué— exigía abrir el archivo, filtrar a mano y confiar en que la copia consultada fuera la vigente.

La propuesta fue sustituir ese flujo por un tablero web con base de datos relacional: una sola fuente de verdad, consultable por varias personas a la vez, con control de acceso por rol y trazabilidad de quién decide qué.

1.1 Qué se construyó

Componente	Tecnología	Función
Base de datos	MySQL 8.4	9 tablas relacionales, 16 migraciones versionadas
API	PHP 8.5 sobre Apache	29 endpoints REST, sin dependencias externas
Interfaz	React 19 + Vite	27 componentes, dos vistas y pantallas de administración
Autenticación	Tokens firmados con HMAC-SHA256	Tres roles: administrador, revisor y consulta

1.2 Estado en producción

Indicador	Valor
Organizaciones en el padrón	779
Apoyos de inversión social 2023-2024	598 (596 vinculados a una OSC: 99.7 %)
Municipios con al menos una OSC	35 de 51
Organizaciones con ubicación en el mapa	605 (77.7 %)
Líneas de código	≈ 13,580 en 124 archivos

El sistema está desplegado y accesible: la interfaz en **osc-nl-dashboard.vercel.app** y la API en **osc-nl.duckdns.org**, ambas sobre HTTPS.

1.3 Qué resuelve, en concreto

- **Padrón consultable.** Las 779 organizaciones con filtros combinables por municipio, rubro, estatus de operación, avance de expediente, resolución y responsable.
- **Expediente digital.** Carga de documentos en PDF o imagen, con aprobación o rechazo por documento y motivo obligatorio al rechazar.
- **Resolución del Registro.** Aceptar o denegar una organización completa, firmada con nombre y fecha de quien decidió.
- **Reparto de trabajo.** Asignación de un responsable por organización, individual o por lote sobre el filtro activo.
- **Importación desde CSV.** Actualización del padrón desde el archivo que ya usa la Secretaría, con vista previa antes de escribir.
- **Indicadores y mapas.** Dos vistas de análisis con gráficas, mapa de calor por municipio y mapa de ubicaciones.

2. Pasos seguidos y cómo se realizaron

El desarrollo siguió siete etapas. Cada una se describe con lo que se hizo, cómo se hizo y —cuando hubo— el problema que obligó a corregir el rumbo, porque esas correcciones explican por qué el sistema quedó como quedó.

2.1 Modelado de datos y migraciones

Se diseñó un esquema relacional normalizado en lugar de replicar la hoja de cálculo. La tabla **OSC** es el centro; alrededor están Municipio, Documentacion, Apoyos, Usuario, Representante_Legal, Beneficiarios, Fuente_Financiamiento y Transparencia.

El esquema no se escribió de una sola vez: se construyó en **16 migraciones numeradas**, cada una un archivo SQL con su justificación en comentarios. Esto permite reconstruir la base desde cero en orden y deja registro de por qué existe cada campo.

Migración	Qué agrega	Por qué
001-007	Esquema inicial	Padrón, municipios, documentación, beneficiarios
008-009	Apoyos y contacto	Inversión social 2023-2024 y datos de contacto
010-011	Revisión y usuarios	Estatus 'Rechazado' y cuentas con rol
012	22 campos del padrón real	Alinear el modelo con las 72 columnas del Excel
013	Resolución del Registro	Aceptar o denegar la OSC completa
014	Origen de la coordenada	Distinguir la verificada de la calculada
015	Responsable asignado	Repartir el padrón entre quienes revisan
016	Ubicación del expediente físico	El archivo en papel sigue existiendo

2.2 Carga inicial del padrón

Las 779 organizaciones se importaron desde los CSV que entregó el socio formador. La carga se hizo con scripts SQL generados a partir de los archivos, ejecutados dentro de una transacción.

Incidente y corrección: la primera importación corrompió todos los acentos. El cliente de MySQL se conecta en latin1 por omisión, y los datos UTF-8 quedaron doblemente codificados. Se diagnosticó comparando la representación hexadecimal —C383 C2AD en lugar de C3AD para la 'í'— y se corrigió forzando default-character-set=utf8mb4 en la conexión, para después reimportar. Una verificación posterior con LIKE '%Ã%' dio un falso positivo en 778 de 779 filas: la colación utf8mb4_0900_ai_ci ignora acentos, así que la comprobación tuvo que hacerse también sobre el hexadecimal.

2.3 API en PHP

Se construyeron 29 endpoints REST que devuelven JSON. La decisión de fondo fue **no usar framework ni dependencias**: el proyecto será entregado a la Secretaría y operado por su propio personal, y un backend sin Composer ni paquetes externos no acumula vulnerabilidades por dependencias sin actualizar ni exige conocer un framework para mantenerlo. El backend completo pesa menos de 100 KB.

Toda la lógica compartida vive en cuatro archivos de configuración: la conexión, el envoltorio de la API, las reglas de negocio y la autenticación. Los endpoints solo arman su consulta y responden.

2.4 Interfaz en React

La interfaz se organizó en dos vistas —Operativa, para trabajar expediente por expediente; y Estratégica, para ver agregados— más las pantallas de administración. Las dos vistas se cargan bajo demanda: entre ambas arrastran la librería de gráficas, que es la mitad del peso de la aplicación, y quien todavía no inicia sesión no necesita descargarla.

2.5 Despliegue del ambiente de demostración

Para desarrollar y mostrar el sistema funcionando se levantó un ambiente en la nube. **No es la infraestructura de entrega**: el socio formador operará el tablero en su propio servidor, y este ambiente se apagará tras el traspaso. Se documenta porque explica las decisiones de configuración que sí se heredan.

Capa	Servicio	Detalle
Base de datos	Amazon RDS (MySQL 8.4.9)	Cifrado en reposo, respaldos automáticos
API	Amazon EC2 (Amazon Linux 2023)	Apache + PHP 8.5, IP elástica
Interfaz	Vercel	Despliegue automático desde GitHub
Dominio y TLS	DuckDNS + Let's Encrypt	HTTPS con renovación automática

El despliegue del backend se automatizó en un script que sincroniza el código por rsync, ajusta permisos y recarga Apache verificando antes la configuración.

El grupo de seguridad de la base solo acepta conexiones desde la IP del equipo de desarrollo, no desde cualquier origen. Es la decisión correcta en seguridad y tiene un costo operativo real: cada vez que cambia la IP del desarrollador hay que actualizar la regla. Ocurrió cuatro veces durante el proyecto.

2.6 Seguridad y control de acceso

- **Consultas preparadas siempre.** Ningún valor de entrada se concatena en SQL. Los marcadores se enlazan con tipo explícito y las consultas preparadas son nativas (ATTR_EMULATE_PREPARES en falso), no simuladas por el driver.

- **Tokens firmados, no sesiones.** La interfaz es estática: cualquier credencial en su código sería legible. La contraseña se verifica en el servidor, que devuelve un token firmado con HMAC-SHA256 y vigencia de 12 horas.
- **Revocación inmediata.** El token es autocontenido, así que seguiría siendo válido aunque se desactive la cuenta. Por eso se comprueba contra la base en cada petición.
- **Roles verificados en el servidor.** Esconder un botón no es proteger: cada endpoint declara qué roles lo pueden invocar.
- **Archivos fuera del DocumentRoot.** Los documentos subidos se guardan con nombre aleatorio y sin extensión, fuera del directorio que sirve Apache, y solo se entregan a través de PHP con verificación de sesión.
- **Credenciales solo en el entorno.** Ninguna contraseña vive en el repositorio; el .gitignore protege los archivos .env y los datos personales del padrón nunca se subieron a control de versiones.

2.7 Funcionalidad operativa

Sobre la base consultable se construyó el flujo de trabajo real del Registro: expediente digital con aprobación por documento, resolución de la organización completa, avance medido contra los 16 documentos obligatorios del padrón, asignación de responsables y registro de dónde está el expediente en papel.

Una decisión de diseño con consecuencia directa: el indicador de completitud documental dividía entre los documentos *entregados*. Con esa fórmula, la primera organización que subiera un documento aprobado habría puesto el tablero en 100 % de completitud para todo el padrón. Se corrigió para dividir entre lo *requerido*: $779 \text{ organizaciones} \times 16 \text{ documentos} = 12,464$.

2.8 Geolocalización

El padrón no traía coordenadas, así que se calcularon a partir de la dirección escrita usando el geocodificador de OpenStreetMap. Una primera prueba dio un resultado inaceptable: de 10 direcciones, 8 encontradas pero solo 6 dentro del municipio correcto, sin forma de saber cuáles eran las malas.

El problema no era la herramienta sino el método. Se corrigieron dos cosas: se pasó de una consulta de texto libre a una consulta estructurada —calle y ciudad en campos separados—, y se validó cada resultado comparándolo contra el municipio que el propio OpenStreetMap reporta para esa coordenada. Con ambas correcciones, la cobertura subió a 93 % en la muestra y **ninguna quedó en el municipio equivocado**.

Sobre las 779 organizaciones: 605 ubicadas, 5 descartadas por caer fuera de su municipio y 169 sin resultado. Las calculadas se dibujan con un símbolo distinto y se declaran como aproximadas, porque no son el domicilio verificado.

3. Código, explicado paso a paso

Se muestran seis fragmentos que concentran las decisiones importantes. Cada uno se explica en términos de qué problema evita.

3.1 Conexión a la base: consultas preparadas reales

```
$pdo = new PDO(
    "mysql:host=$host;dbname=$dbname;charset=utf8mb4",
    $user, $password,
    [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ]
);
```

- **charset=utf8mb4** en la cadena de conexión. Es lo que faltaba en la importación inicial y provocó la corrupción de acentos.
- **ERRMODE_EXCEPTION**: un error de SQL lanza una excepción en vez de devolver false en silencio y seguir como si nada.
- **EMULATE_PREPARES en false**: las consultas preparadas se ejecutan en el servidor de MySQL, no las simula el driver interpolando cadenas. Es lo que hace que la separación entre consulta y datos sea real y no una convención.

3.2 Reglas de negocio en un solo lugar

El padrón obliga a tomar decisiones interpretativas: qué cuenta como donataria vigente, cómo se resume el estatus de varios documentos, cómo se agrupan 82 rubros en categorías legibles. Esas reglas viven en un archivo, documentadas como supuestos, y no repartidas por los endpoints.

```
// El estatus del expediente es el PEOR de sus documentos: si uno fue
// rechazado, la organización tiene un problema aunque los demás estén bien.
const SQL_ESTATUS_DOCUMENTAL =
    "CASE
        WHEN COUNT(d.id_documento) = 0 THEN 'Pendiente'
        WHEN SUM(d.estatus_validacion = 'Rechazado') > 0 THEN 'Rechazado'
        WHEN SUM(d.estatus_validacion = 'Vencido') > 0 THEN 'Vencido'
        WHEN SUM(d.estatus_validacion = 'Pendiente') > 0 THEN 'Pendiente'
        ELSE 'Completo'
    END";
```

La misma constante alimenta la columna de la tabla, el filtro y los indicadores. Sin eso, la tarjeta del tablero y las filas podrían contradecirse y nadie sabría cuál creer.

3.3 Autenticación por token firmado

La interfaz es un sitio estático. Cualquier credencial dentro de su código sería legible por quien abra las herramientas del navegador: no sería autenticación, sería un candado con la llave pegada. La contraseña vive solo en el servidor, como hash bcrypt.

```
function crearToken(array $persona): array
{
    $horas = (int) (env('AUTH_HORAS_VIGENCIA', '12') ?: 12);
    $expira = time() + $horas * 3600;

    $carga = b64url(json_encode([
        'id'      => $persona['id'],
        'usuario' => $persona['usuario'],
        'rol'     => $persona['rol'],
        'exp'     => $expira,
    ], JSON_UNESCAPED_UNICODE));

    $firma = b64url(hash_hmac('sha256', $carga, secretoApp(), true));

    return ['token' => "$carga.$firma", 'expira' => $expira];
}
```

- El token lleva identidad, rol y expiración, firmados con una clave secreta del servidor. Alterar el contenido invalida la firma.
- No se guarda en la base: se valida verificando la firma, así que no hay estado de sesión que mantener ni limpiar.
- La verificación usa **hash_equals**, que compara en tiempo constante para no filtrar información por el tiempo de respuesta.

3.4 Importación del padrón en dos pasos

Importar es una escritura masiva sobre datos reales. El endpoint funciona en dos fases: sin confirmar, analiza el archivo y reporta qué haría; solo con confirmación explícita escribe.

```
// Solo se tocan las columnas que el archivo trae con valor. Un campo
// vacío en el CSV no borra lo que ya estaba capturado.
$asignaciones = [];
foreach ($osc as $c => $v) {
    if ($v !== null && $c !== 'no_registro') {
        $asignaciones[] = "$c = :$c";
    }
}
```

- **Actualiza, no reemplaza.** Las organizaciones que estén en la base pero no en el archivo se quedan intactas: importar no da de baja a nadie.

- **Empareja por folio** (el identificador del Registro), lo que hace la operación idempotente: subir dos veces el mismo archivo actualiza en vez de duplicar el padrón.
- **Todo en una transacción.** Si algo falla a la mitad, no queda un padrón medio actualizado que nadie sabe en qué estado está.

3.5 Geocodificación validada contra el municipio

El fragmento que evita poner puntos equivocados en el mapa. Se revisan varias claves porque OpenStreetMap clasifica distinto según el tamaño de la localidad.

```
function coincideMunicipio(array $resultado, string $municipioPadron): bool
{
    $dir = $resultado['address'] ?? [];
    if (normalizar($dir['state'] ?? '') !== normalizar(ESTADO)) return false;

    $esperado = normalizar($municipioPadron);
    foreach (['city', 'town', 'village', 'municipality', 'county', 'state_district'] as $clave) {
        if (isset($dir[$clave]) && normalizar($dir[$clave]) === $esperado) return true;
    }
    return false;
}
```

Lo que no coincide se descarta. El criterio de fondo: un mapa con puntos mal ubicados que se ven igual que los buenos es peor que un mapa incompleto, porque nadie sabe cuáles creer. Se probó primero validar contra los polígonos municipales del mapa de calor y no sirvió: están simplificados para dibujar —Monterrey tiene 31 vértices— y en el área metropolitana rechazaban puntos correctos.

3.6 Interfaz: carga de datos sin condiciones de carrera

El hook que usan todas las pantallas para pedir datos. El detalle importante está en cómo decide si está cargando.

```
// Identifica de forma única la petición que corresponde al render actual.
const clave = `${endpoint}?${paramsSerializados}#${intentos}`;

// `cargando` se deriva en render en vez de guardarse en estado: si el
// resultado que tenemos no corresponde a la petición actual, seguimos
// esperando. Evita un setState extra (y un render extra) por cada carga.
return {
    datos: resultado.datos,
    error: resultado.clave === clave ? resultado.error : null,
    cargando: resultado.clave !== clave,
    recargar,
};
```

Cada petición se identifica con la combinación de endpoint, parámetros e intento. Si alguien cambia un filtro mientras la respuesta anterior viaja, la respuesta vieja no puede sobrescribir a la nueva: su clave ya

no coincide. Además cada petición se cancela al desmontar el componente.

4. Interfaces de usuario

Las capturas siguientes muestran el sistema en funcionamiento con los datos reales del padrón.

Pendiente: colocar las capturas de pantalla en la carpeta entregables/capturas/ y volver a generar el documento.

5. Mantenimiento y evolución futura

Esta sección es para el cliente. Distingue tres cosas que suelen mezclarse: lo que hay que hacer para que el sistema siga funcionando, lo que ya se sabe que está incompleto, y lo que conviene construir después.

5.1 Mantenimiento operativo

Estas tareas corresponden al sistema instalado en el servidor de la Secretaría. Varias de ellas las resolvía sola la infraestructura de nube del ambiente de demostración; en servidor propio hay que programarlas explícitamente, y esa es justamente la diferencia que más se pasa por alto en un traslado.

Tarea	Frecuencia	Qué pasa si no se hace
Ejecutar el respaldo de la base y del directorio de documentos	Diaria (programada)	Una falla de disco o un borrado accidental se vuelven irreversibles
Verificar que el respaldo existe y no está vacío	Semanal	Un cron que dejó de correr no avisa: se descubre el día que se necesita restaurar
Probar una restauración completa en un ambiente aparte	Semestral	Un respaldo que nunca se restauró es una suposición, no una garantía
Vigilar el espacio libre en disco	Mensual	Los documentos crecen sin límite; con el disco lleno, la base deja de aceptar escrituras
Revisar el registro de errores de PHP y del servidor web	Mensual	Los fallos intermitentes pasan inadvertidos hasta que se vuelven permanentes
Comprobar los permisos del archivo de configuración con credenciales	Tras cada cambio en el servidor web	Un archivo legible por todos expone la contraseña de la base de datos
Actualizar paquetes del sistema operativo	Trimestral	Vulnerabilidades conocidas sin parchar
Renovar el certificado TLS	Antes de su vencimiento	El navegador bloquea el acceso y el tablero queda inutilizable
Rotar la clave de firma de tokens (APP_SECRET)	Anual o ante sospecha	Un secreto filtrado permite fabricar sesiones válidas de cualquier persona
Revisar cuentas activas y desactivar las que ya no operan	Semestral	Quien dejó la dependencia conserva acceso al padrón

Sobre los permisos del archivo de configuración: las credenciales de la base viven en la configuración del servidor web, no en el código. Cualquier herramienta que regenere esa configuración puede devolverle permisos de lectura para todos los usuarios del sistema. En el ambiente de demostración ocurrió con Certbot al renovar el certificado, y el archivo con la contraseña quedó legible. Convenga o no usar esa herramienta en el servidor institucional, la lección se traslada: después de tocar la configuración del servidor web, verificar que ese archivo siga en modo 600.

5.2 Lo que se sabe incompleto

Pendiente	Impacto hoy	Cómo se resuelve
El padrón completo de 72 columnas no se ha importado	Cuatro funciones construidas están vacías: estatus de operación, coordenadas verificadas, avance de expediente y beneficiarios	Solicitar a la Secretaría el export en CSV y cargarlo desde la pantalla de importación
605 coordenadas son aproximadas	El mapa ubica en la cuadra, no en la puerta	Se reemplazan solas al importar las coordenadas del padrón, que tienen prioridad
Sin coordenada quedan 174 organizaciones	No aparecen en el mapa de ubicaciones	Capturar la coordenada en la siguiente visita domiciliaria
Las tablas de beneficiarios, financiamiento y transparencia están vacías	La Vista Estratégica muestra ceros en esos indicadores	Llegan con la importación del padrón completo

5.3 Riesgos a vigilar

- **Dependencia de mosaicos externos (aplica al traslado).** El mapa de ubicaciones pide las imágenes del mapa a los servidores de OpenStreetMap. Si el servidor del socio formador no tiene salida a internet, el mapa quedará en blanco: los puntos se dibujarán sobre un fondo vacío. Es resoluble instalando un servidor de mosaicos propio, pero hay que preverlo antes de migrar, no después.
- **Punto único de falla.** La API y la base de datos corren en un solo servidor. Si ese servidor se cae, el tablero deja de responder por completo. Para el uso previsto puede ser aceptable, pero conviene que el área de sistemas de la Secretaría lo decida a conciencia y no por omisión: en cuanto el Registro dependa del tablero para su operación diaria, una caída deja de ser una molestia y pasa a detener el trabajo.
- **Coordenadas aproximadas mal interpretadas.** Se dibujan distinto y lo declaran al abrirlas, pero conviene insistir en la capacitación: no son domicilios verificados y no deben usarse como única referencia para una visita.
- **Concentración del conocimiento.** El proyecto lo desarrolló un equipo académico. La documentación técnica está en el repositorio y el código está comentado en español explicando el porqué de cada decisión, pero conviene una sesión de traspaso con quien vaya a operarlo.

5.4 Recomendaciones al cliente

Inmediatas

- **Solicitar el export del padrón con las 72 columnas.** Es lo que más desbloquea y lo que más tarda en conseguirse. Todo lo demás puede esperar; esto no.
- **Crear las cuentas del personal que revisa,** con rol Revisor. Mientras no existan, el reparto de trabajo no se puede usar y toda la auditoría queda a nombre de una sola persona.
- **Repartir el padrón** con la asignación por lote, por municipio o por zona, según cómo se organice el equipo.

A mediano plazo

- **Definir una política de retención documental.** Hoy el sistema guarda todo lo que se sube. Conviene decidir cuánto tiempo se conservan los documentos y qué se hace con los de organizaciones dadas de baja, antes de que el volumen obligue a improvisar.
- **Digitalizar el archivo físico por prioridad.** El campo de ubicación física permite saber dónde está cada expediente en papel; usarlo para planear qué se digitaliza primero —empezando por las organizaciones activas con expediente incompleto— convierte una tarea inabarcable en una lista ordenada.
- **Publicar un subconjunto como datos abiertos.** El padrón sin datos de contacto es información de interés público y ya está estructurado; publicarlo tiene un costo marginal bajo y un beneficio de transparencia alto.

Antes de un traspaso definitivo

El socio formador operará el sistema en **su propio servidor**, no en la nube. La infraestructura actual en AWS es un ambiente de demostración: sirvió para desarrollar y mostrar el sistema funcionando, y se apagará tras la entrega. El capítulo 6 contiene la guía de instalación.

- **Rotar todas las credenciales tras el traspaso:** contraseña de la base de datos, clave de firma de tokens (APP_SECRET) y contraseñas de las cuentas. Quien desarrolló no debe conservar acceso al sistema en producción.
- **Migrar los datos, no solo el código.** El padrón vive en la base de datos, y los documentos subidos en el sistema de archivos. Un respaldo del código sin los otros dos entrega un sistema vacío.
- **Dar de baja el ambiente de demostración** una vez verificada la instalación propia: dejar dos copias en línea garantiza que en algún momento alguien consulte la equivocada.
- **Acordar quién mantiene el sistema.** El código está comentado en español explicando el porqué de cada decisión y la documentación técnica va en el repositorio, pero el conocimiento operativo —cómo respaldar, cómo restaurar, qué revisar— necesita una persona designada dentro de la Secretaría.

6. Guía de instalación en servidor propio

El sistema se entrega para operar en la infraestructura del socio formador. Nada del código está atado a Amazon: **toda la configuración que depende del entorno vive en variables** —credenciales de base de datos, clave de firma, ruta de archivos, orígenes permitidos— y ninguna está escrita dentro del código. Trasladarlo es instalar los requisitos, restaurar los datos y definir esas variables.

6.1 Requisitos del servidor

Componente	Versión mínima	Notas
PHP	8.1	Probado en 8.5. Requiere las extensiones PDO_MySQL, mbstring y fileinfo
MySQL	8.0	Probado en 8.4.9. Debe soportar utf8mb4 y funciones de ventana
Servidor web	Apache 2.4 o Nginx	Con soporte para PHP y cabeceras personalizadas
Espacio en disco	1 GB inicial	Crece con los documentos: estimar 2 MB por expediente
Node.js	20 o superior	Solo para compilar la interfaz; no hace falta en el servidor

La interfaz compilada son **archivos estáticos**: aproximadamente 1 MB de HTML, JavaScript y CSS. No requiere Node.js en el servidor, solo un directorio servido por el servidor web. Puede convivir con el sitio institucional existente.

6.2 Procedimiento de instalación

Paso 1 — Restaurar la base de datos

En la entrega se incluye un volcado completo. Si se prefiere partir de un esquema limpio, las 16 migraciones se aplican en orden y después se importa el padrón.

```
# Crear la base con la codificación correcta. Es importante: sin utf8mb4
# los acentos del padrón se corrompen al importar.
CREATE DATABASE osc_nl
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

# Restaurar el volcado. El parámetro de codificación NO es opcional:
# el cliente de MySQL usa latin1 por omisión.
mysql --default-character-set=utf8mb4 -u usuario -p osc_nl < respaldo.sql

# Crear un usuario sin privilegios de administración para la aplicación.
CREATE USER 'osc_app'@'localhost' IDENTIFIED BY '<contraseña-nueva>';
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON osc_nl.* TO 'osc_app'@'localhost';
```

La aplicación no necesita permisos para crear ni borrar tablas: solo lee y escribe filas. Darle un usuario con menos privilegios limita el daño si alguna vez se encontrara una falla en la API.

Paso 2 — Colocar el backend

```
# El código de la API, en el directorio que sirve el servidor web.
/var/www/osc-api/

# El directorio de documentos va FUERA de ese árbol. Es deliberado:
# si el servidor web pudiera servirlos directamente, cualquiera con la
# URL leería actas y comprobantes sin iniciar sesión.
mkdir -p /var/osc-archivos
chown www-data:www-data /var/osc-archivos
chmod 750 /var/osc-archivos
```

Paso 3 — Definir las variables de entorno

Se definen en la configuración del servidor web (con SetEnv en Apache, o fastcgi_param en Nginx), no en un archivo dentro del directorio público. Ese archivo de configuración debe tener permisos 600: contiene la contraseña de la base.

Variable	Qué es
DB_HOST, DB_NAME, DB_USER, DB_PASSWORD	Conexión a la base de datos
APP_SECRET	Clave para firmar los tokens de sesión. Generar una nueva, larga y aleatoria
ARCHIVOS_DIR	Ruta del directorio de documentos (fuera del árbol público)
CORS_ORIGENES	Dominio desde el que se sirve la interfaz. Sin este valor acepta cualquier origen
AUTH_USUARIO, AUTH_PASSWORD_HASH	Cuenta de arranque, solo para crear la primera cuenta real
AUTH_HORAS_VIGENCIA	Duración de la sesión. Por omisión, 12 horas

```
# Generar una clave de firma nueva (NO reutilizar la del ambiente de demostración)
php -r "echo bin2hex(random_bytes(32));"

# Generar el hash de la contraseña de arranque
php -r "echo password_hash('la-contrasena', PASSWORD_BCRYPT);"
```

Paso 3b — Ajustar el límite de subida de archivos

El expediente digital acepta documentos de hasta 10 MB, pero PHP viene configurado de fábrica en 2 MB. Sin este ajuste, la pantalla promete un límite que el servidor rechaza, y el mensaje de error contradice lo

que la persona acaba de leer.

```
# Archivo aparte, no editando php.ini: así una actualización de PHP
# no se lleva el cambio por delante.
/etc/php.d/99-osc-tablero.ini

upload_max_filesize = 10M
post_max_size = 12M

sudo systemctl restart php-fpm
```

Dos advertencias sobre este ajuste. Primera: `post_max_size` va POR ENCIMA de `upload_max_filesize`, porque el cuerpo de la petición lleva el archivo más los otros campos del formulario; con los dos iguales, un archivo de exactamente 10 MB se rechazaría. Segunda: si PHP corre como FPM —lo habitual—, poner estos valores como `php_value` en la configuración de Apache NO funciona: se ignoran en silencio, que es peor que fallar, porque parece configurado y no lo está.

Paso 4 — Compilar y publicar la interfaz

La dirección de la API queda **incrustada al compilar**, no se lee en tiempo de ejecución. Por eso hay que compilar apuntando al dominio definitivo: si se copian los archivos del ambiente de demostración, seguirán llamando al servidor de AWS.

```
cd frontend

# Apuntar a la API del servidor institucional
echo "VITE_API_URL=https://tablero.ejemplo.gob.mx/api" > .env.production

npm install
npm run build

# El resultado, en el directorio que sirve el servidor web
cp -r dist/* /var/www/tablero/
```

La interfaz usa rutas del navegador (por ejemplo `/usuarios`), así que el servidor web debe entregar `index.html` para cualquier ruta que no corresponda a un archivo. En Apache se resuelve con `FallbackResource /index.html`; en Nginx, con `try_files`.

Paso 5 — Crear la primera cuenta

```
php /var/www/osc-api/herramientas/crear-usuario.php
```

Pide la contraseña sin mostrarla en pantalla y sin dejarla en el historial de comandos. La primera cuenta debe ser de rol Administrador: al crearla, el acceso de arranque por variables de entorno deja de funcionar, y sin un administrador nadie podría volver a entrar a la administración.

6.3 Diferencias respecto al ambiente de demostración

Función	En la demostración	En servidor propio
RespalDOS	Automáticos de Amazon RDS	Hay que programarlos: mysqldump diario más copia del directorio de documentos
Certificado TLS	Let's Encrypt con renovación automática	Certificado institucional, o autoridad certificadora interna si la red es cerrada
Interfaz	Vercel, despliegue automático al subir el código	Copiar los archivos compilados al servidor web
Mapa de ubicaciones	Mosaicos de OpenStreetMap por internet	Requiere salida a internet, o instalar un servidor de mosaicos propio
Alta disponibilidad	Una sola instancia	Según la política del área de sistemas de la Secretaría

El respaldo es la diferencia más importante y la más fácil de pasar por alto. En la nube ocurría solo; en servidor propio, si nadie lo programa, no existe. Un respaldo completo son dos cosas: el volcado de la base de datos y una copia del directorio de documentos. Respaldar solo una de las dos deja el sistema irrecuperable.

```
# Ejemplo de respaldo diario (cron, 2 de la madrugada)
0 2 * * * mysqldump --default-character-set=utf8mb4 \
    --single-transaction osc_nl \
    | gzip > /respaldos/osc_nl_$(date +%F).sql.gz

0 3 * * * tar czf /respaldos/archivos_$(date +%F).tar.gz /var/osc-archivos
```

--single-transaction toma el volcado sin bloquear las tablas, de modo que el respaldo no interrumpe a quien esté usando el tablero. Y conviene verificar la restauración al menos una vez: un respaldo que nunca se probó es una suposición, no una garantía.

7. Anexo: inventario técnico

7.1 Endpoints de la API

Grupo	Endpoints
Padrón	osc, osc-detalle, filtros, osc-ubicaciones
Indicadores	kpis-operativos, kpis-estrategicos, kpis-apoyos, registros-por-mes, distribucion-rubro, densidad-municipio, beneficiarios-por-edad, fuentes-financiamiento
Apoyos	apoyos, apoyos-por-tipo, apoyos-por-poblacion
Expediente	documentos, documento-subir, documento-archivo, documento-revisar
Flujo de revisión	osc-revisar, osc-asignar, osc-operacion, osc-ubicacion-fisica
Acceso	login, sesion, cambiar-password, usuarios, usuario-guardar
Importación	padron-importar

7.2 Comandos de operación

```
# 1. Base de datos: aplicar migraciones (una vez, base vacía)
cd database && ./run_migrations.sh

# 2. Backend: sincroniza, ajusta permisos y recarga Apache
./backend/deploy/subir.sh ec2-user@<ip-del-servidor> ~/.ssh/llave.pem

# 3. Interfaz: Vercel despliega solo al subir a la rama main
git push origin main

# Recuperación: crear o restablecer una cuenta desde la línea de comandos
php backend/herramientas/crear-usuario.php
```